Donor Portal API Reference

This document defines the donor-facing endpoints: the skippable onboarding wizard, the personal profile, making a donation through Stripe Checkout, the Stripe webhook that records it, the donor's donation history and totals, and account closure. It follows the same conventions established in 07-Public API Reference — see that document for base URL, standard response structure, and error code definitions, which are not repeated here. Every response uses the standard envelope: `{ success, message, data, pagination? }` on success and `{ success: false, message, error: { code, details } }` on failure. Every endpoint in this document is Donor-only except the Stripe webhook (5.2), which Stripe calls. A donor is identified via `req.user.userID`. All example data in this document is fictional.

1. Overview

The donor portal is where a supporter gives money to a shelter and keeps track of what they have given. Registration and login are documented in 06-Auth_System; this document covers everything a donor does after that point. It spans five areas:

- **Optional onboarding** — a short wizard (personal details, address, review) that a donor may skip; donating never waits on it (Sections 3 and 4).
- **Profile** — contact details and address, kept on the donor's own record (4.1–4.2).
- **Donating through Stripe Checkout** — the API returns a Checkout URL; the donation itself is recorded by the Stripe webhook after payment (Section 5).
- **Donation history and totals** — the confirmation poll, a paginated history, lifetime and this-year totals per shelter, and one donation in detail (Section 6).
- **Closing the account** — deactivate or permanently delete; every donation stays on the shelter's books (7.1 and Section 9).

The table below lists every endpoint in this document. Paths are relative to `/api/v1`, whose base URL is:

```
Development:  http://localhost:5000/api/v1
Production:   https://petpals-api.onrender.com/api/v1
```

Donors have no Pending state, so every Donor route uses plain `authenticate`; there is no allowPending column. The staff and public endpoints that touch donations are summarised in Section 10, and their full behaviour remains in 11-Staff API Reference and 07-Public API Reference.

Endpoint	Roles	Purpose
GET /donors/me	Donor	Own profile (4.1)
PUT /donors/me	Donor	Update own profile (4.2)
PATCH /donors/me/onboarding-step	Donor	Advance the onboarding wizard (4.3)
PATCH /donors/me/onboarding-complete	Donor	Finish onboarding (4.4)
POST /donations/checkout	Donor	Start a Stripe Checkout (5.1)
POST /webhooks/stripe	Stripe (signed)	Record a paid donation (5.2)
GET /donors/me/donations	Donor	Confirmation poll, or history (6.1)
GET /donors/me/donations/stats	Donor	Totals and per-shelter breakdown (6.2)
GET /donors/me/donations/:id	Donor	One donation in detail (6.3)
DELETE /donors/me	Donor	Deactivate or delete own account (7.1)

> ***Never returned:*** *userPassword, refreshToken, stripeCustomerID, stripeCheckoutSessionID and stripePaymentIntentID are excluded from every response in this document, regardless of role — this mirrors the NF-10 data privacy requirement.*

2. Donor Lifecycle & Access Rules

A donor has two account states. Unlike Volunteers, Vets and Staff, a donor needs no approval and submits no government ID.

State	Can log in	API access	How the donor gets here
Active	Yes	All donor routes	Registration (2.1)
Deactivated	No — rejected at login	None; an already-issued access token stops working immediately (2.3)	Self-deactivated (7.1)

2.1 Registration

A donor registers through the standard registration endpoint (see 06-Auth_System) with role "donor". No `shelterID` is sent — a donor belongs to the network, not to a shelter. The account is created Active immediately: there is no approval step and no government ID.

2.2 Onboarding never gates access

The login and refresh responses include `accountStatus`, `onboardingComplete` and `onboardingStep`, which the frontend uses to decide whether to show the wizard. Onboarding is skippable (Section 3), so none of the routes in this document checks `onboardingComplete`: a donor who has skipped can call every endpoint, including POST /donations/checkout.

2.3 Live account-status enforcement

Account status is checked live on every protected request, not only at login. Deactivation therefore takes effect immediately: a deactivated donor's already-issued access token is rejected on its next use.

2.4 Existence masking: 404 instead of 403

A donor must not be able to discover which donations exist, or who made them. Where a resource belongs to a different donor, the API answers exactly as if it did not exist.

Situation	Response
Another donor's donation on GET /donors/me/donations/:id (6.3)	404 NOT_FOUND — identical to a donation that does not exist
Staff, Admin or another role calling a donor-only route	403 FORBIDDEN
Missing or invalid token, or a Deactivated account	401 UNAUTHORIZED

3. Onboarding Wizard

A new donor is offered a three-step wizard. Registration counts as step 1 and happens before the wizard. At each step the client first saves that step's data, then calls `PATCH /donors/me/onboarding-step` with the step just completed. The client resumes the wizard from the `onboardingStep` returned at login.

Step	Screen	Data saved first	Then call
1	Registration	Account created at registration (not part of the wizard)	—
2	Personal	donorName, donorPhone, donorDOB, donorSex, avatarSeed via PUT /donors/me (4.2)	PATCH /donors/me/onboarding-step { step: 2 }

Step	Screen	Data saved first	Then call
3	Address	addressLine1, addressLine2, city, state, zip, country via PUT /donors/me (4.2)	PATCH /donors/me/onboarding-step { step: 3 }
4	Review	Nothing new — the donor reviews and submits	PATCH /donors/me/onboarding-complete (4.4), which sets onboardingStep to 4

`onboardingStep` only ever advances: the server sets it to the larger of its current value and step + 1, capped at 4, so going back in the wizard or repeating a call never moves the donor backward.

Skipping: the wizard offers "Skip for now". Skipping is a client-side choice that lasts for the current browser session — it calls no endpoint and changes nothing on the server. The donor can use the dashboard and donate for that session. Because `onboardingComplete` is still false, the next login brings them back to the wizard, at the `onboardingStep` they reached. This is the same behaviour as the adopter wizard.

Required for completion: donorPhone, donorDOB, donorSex, addressLine1, city, state, zip and country. addressLine2 is optional. Completion unlocks nothing: donating never waits on onboarding.

4. Profile & Onboarding Endpoints

These endpoints let the logged-in donor manage their own profile and onboarding. All of them identify the donor via `req.user.userID`, and none checks onboardingComplete.

4.1 GET /api/v1/donors/me

Returns the logged-in donor's own profile.

Request

Field	Value
Method	GET
Auth required	Yes — Donor only
Path/Body fields	None — donor identified via req.user.userID

Example request

```
GET /api/v1/donors/me
Authorization: Bearer <accessToken>
```

Response

```
200 OK
{
  "success": true,
  "message": "Profile retrieved successfully",
  "data": {
    "userID": 31,
    "avatarSeed": "a1b2c3d4-...",
    "donorName": "Riley Chen",
    "donorPhone": "+15555550188",
    "donorDOB": "1990-03-14",
    "donorSex": "F",
    "addressLine1": "100 Example Street",
    "addressLine2": null,
    "city": "Athens",
    "state": "GA",
```

```
      "zip": "30601",
      "country": "USA",
      "createdAt": "2026-09-01T10:00:00Z",
      "accountStatus": "Active",
      "onboardingComplete": true,
      "onboardingStep": 4,
      "emailVerified": true,
      "lastLoginAt": "2026-10-03T08:30:00Z",
      "user": { "userEmail": "riley.chen@example.com" }
    }
  }
```

`donorPhone` is E.164 and `donorSex` is M, F or O. `accountStatus` is Active or Deactivated. `emailVerified` and `lastLoginAt` come from the USERS table, not the DONOR table, and `user.userEmail` is the login email. Passwords, refresh tokens and `stripeCustomerID` are never included.

Error Cases

Scenario	Error Code	HTTP Status
No/invalid token, or the account is Deactivated	UNAUTHORIZED	401
Role not permitted	FORBIDDEN	403
Donor record does not exist	NOT_FOUND	404

4.2 PUT /api/v1/donors/me

Partially updates the logged-in donor's own profile. Only fields present in the body are changed.

Request

Field	Value
Method	PUT
Auth required	Yes — Donor only
Body fields	Any subset of: avatarSeed, donorName (≤45 chars), donorPhone, donorDOB (date), donorSex (M / F / O), addressLine1, addressLine2, city, state, zip, country. donorPhone, donorDOB and donorSex accept null to clear them.

`donorPhone` is normalised to E.164 server-side; a number that cannot be parsed is rejected with VALIDATION_ERROR (422). `accountStatus` and `stripeCustomerID` are not editable here — providing either returns 400 listing the rejected field(s), before any write occurs.

Example request

```
PUT /api/v1/donors/me
Authorization: Bearer <accessToken>
Content-Type: application/json
{
  "donorPhone": "(555) 555-0188",
  "donorSex": "F",
  "city": "Athens"
}
```

Response

```
200 OK
{
  "success": true,
  "message": "Profile updated successfully",
  "data": { "...": "same shape as GET /donors/me (4.1)" }
}
```

Error Cases

Scenario	Error Code	HTTP Status
Rejected field present (accountStatus, stripeCustomerID)	BAD_REQUEST	400
Invalid value (donorSex not M/F/O, invalid donorDOB, donorName over 45 chars, wrong type)	BAD_REQUEST	400
No updatable fields provided	BAD_REQUEST	400
No/invalid token, or the account is Deactivated	UNAUTHORIZED	401
Role not permitted	FORBIDDEN	403
Donor record does not exist	NOT_FOUND	404
donorPhone cannot be parsed to E.164	VALIDATION_ERROR	422

4.3 PATCH /api/v1/donors/me/onboarding-step

Advances the logged-in donor through the onboarding wizard. Called after a step's own data has been saved (see Section 3).

Request

Field	Value
Method	PATCH
Auth required	Yes — Donor only
Body fields	step (int, required, 2–4) — the wizard step just completed

Advance-only: the server sets `onboardingStep = min(max(currentStep, step + 1), 4)`. A lower or equal step is ignored rather than moving the donor backward, and the value is capped at 4.

Example request

```
PATCH /api/v1/donors/me/onboarding-step
Authorization: Bearer <accessToken>
Content-Type: application/json
{ "step": 3 }
```

Response

```
200 OK
{
  "success": true,
  "message": "Onboarding step updated successfully",
  "data": { "...": "same shape as GET /donors/me (4.1), onboardingStep: 4" }
}
```

Error Cases

Scenario	Error Code	HTTP Status
step missing or not an integer from 2 to 4	BAD_REQUEST	400
No/invalid token, or the account is Deactivated	UNAUTHORIZED	401
Role not permitted	FORBIDDEN	403
Donor record does not exist	NOT_FOUND	404

4.4 PATCH /api/v1/donors/me/onboarding-complete

Marks onboarding complete — the final submit of the wizard's Review step.

Request

Field	Value
Method	PATCH
Auth required	Yes — Donor only
Body fields	None

Requires donorPhone, donorDOB, donorSex, addressLine1, city, state, zip and country. Sets `onboardingComplete` to true and `onboardingStep` to 4. There is no approval afterwards — the donor was already Active.

Example request

```
PATCH /api/v1/donors/me/onboarding-complete
Authorization: Bearer <accessToken>
```

Response

```
200 OK
{
  "success": true,
  "message": "Onboarding completed successfully",
  "data": { "...": "same shape as GET /donors/me (4.1)" }
}
```

Error Cases

Scenario	Error Code	HTTP Status
No/invalid token, or the account is Deactivated	UNAUTHORIZED	401
Role not permitted	FORBIDDEN	403
Donor record does not exist	NOT_FOUND	404
One or more required items are missing — the message names every missing item	CONFLICT	409

Example 409 body when only the phone and ZIP are missing:

```
{
  "success": false,
  "message": "Onboarding is incomplete — missing: Phone, ZIP",
  "error": { "code": "CONFLICT", "details": "..." }
}
```

5. Donating

Donations use the same Stripe Checkout pattern as the adoption application fee. The API never takes card details and never creates the Donation row at checkout: it only returns a Checkout URL. The row is created by the Stripe webhook after payment, from the amount Stripe actually charged.

5.1 POST /api/v1/donations/checkout

Starts a Stripe Checkout for a donation to one shelter and returns the URL to send the donor to.

Request

Field	Value
Method	POST
Auth required	Yes — Donor only
Content-Type	application/json
Body fields	shelterID (int, required), amount (integer, required — whole US dollars, 1–10000), donationDesc (string, optional, ≤300 chars)

The shelter must exist and not be Closed: Open and Full shelters both accept donations. The donor's first donation creates a Stripe customer for them, stored as `stripeCustomerID` and reused afterwards; it is never returned. Nothing here checks `onboardingComplete`. Stripe sends the donor back to one of two pages:

- **Success:** `/donor/donate/confirmation?session_id={CHECKOUT_SESSION_ID}` — the confirmation page polls for the donation (6.1).
- **Cancel:** `/donor/donate` — the donor returns to the Donate page and nothing is recorded.

Example request

```
POST /api/v1/donations/checkout
Authorization: Bearer <accessToken>
Content-Type: application/json
{
  "shelterID": 3,
  "amount": 50,
  "donationDesc": "For the winter blanket drive"
}
```

Response

```
201 Created
{
  "success": true,
  "message": "Checkout session created successfully",
  "data": {
    "checkoutUrl": "https://checkout.stripe.com/c/pay/cs_test_a1b2c3..."
  }
}
```

A 201 means only that a Checkout session exists. No donation is recorded, and history and stats do not change, until the webhook (5.2) runs.

Error Cases

Scenario	Error Code	HTTP Status
shelterID missing or not an integer; amount 0, over 10000, fractional or a string; donationDesc over 300 chars	BAD_REQUEST	400

Scenario	Error Code	HTTP Status
No/invalid token, or the account is Deactivated	UNAUTHORIZED	401
Role not permitted	FORBIDDEN	403
Shelter does not exist	NOT_FOUND	404
Shelter is Closed	CONFLICT	409

5.2 POST /api/v1/webhooks/stripe

The Stripe webhook. Stripe calls it; clients never do. It is the only place a Donation row is created.

Request

Field	Value
Method	POST
Auth required	No bearer token — the request is authenticated by its Stripe signature
Called by	Stripe, for the checkout.session.completed event
Body fields	The raw Stripe event. A donation session carries metadata.kind = "donation".

The event is handled as follows:

Event	Result
Paid session with metadata.kind = "donation"	`finalizeDonation` records one Donation: donationAmt = amount_total / 100 (the amount Stripe actually charged), the donationDesc and shelter chosen at checkout, and the session ID as stripeCheckoutSessionID.
Replayed event for a session already recorded	Ignored — stripeCheckoutSessionID is unique, so no second Donation is created.
Session that is not paid	Nothing is recorded.
Session without a metadata.kind	Not a donation — handled by the adoption-application fee flow as before.
Recording fails	Answers 500 so that Stripe retries the event later.

Because `donationAmt` comes from `amount_total` rather than from the request body, what the donor sees on the Checkout page and what is stored always agree. `stripePaymentIntentID` is stored for reconciliation and, like the session ID, is never returned.

Error Cases

Scenario	Error Code	HTTP Status
Stripe signature missing or invalid	BAD_REQUEST	400
Recording the donation fails (Stripe retries)	Server error	500

6. Donation History & Stats

6.1 GET /api/v1/donors/me/donations

One endpoint with two uses. With `checkoutSessionId` it is the confirmation page’s poll; without it, it is the donor’s paginated history. Only the logged-in donor’s own donations are ever returned.

Request

Field	Value
Method	GET
Auth required	Yes — Donor only
Query params	checkoutSessionId (optional — switches to poll mode), shelterID (optional), dateFrom (optional ISO date-time, inclusive), dateTo (optional ISO date-time, exclusive), page (int, optional, default 1), limit (int, optional, default 20, max 100)

Poll mode. The webhook can land a few seconds after the donor returns from Stripe, so the confirmation page polls with `?checkoutSessionId=` taken from the success URL. `data` is that donation, or `null` until the webhook has recorded it. A not-yet-recorded donation is a normal state, so the answer is 200 with `data: null`, never 404.

History mode. Without `checkoutSessionId`, the donor's donations are returned newest first, optionally narrowed by `shelterID` and by a date range on donationDate (`dateFrom` inclusive, `dateTo` exclusive).

Example request

```
GET /api/v1/donors/me/donations?shelterID=3&dateFrom=2026-01-
01T00:00:00Z&page=1&limit=20
Authorization: Bearer <accessToken>
```

Response (history)

```
200 OK
{
  "success": true,
  "message": "Donations retrieved successfully",
  "data": [
    {
      "donationID": 12,
      "donationCode": "DON-00012",
      "donationDate": "2026-09-14T16:20:00Z",
      "donationAmt": 50.00,
      "donationDesc": "For the winter blanket drive",
      "shelter": { "shelterID": 3, "shelterName": "Athens Shelter" }
    }
  ],
  "pagination": { "page": 1, "limit": 20, "total": 1, "totalPages": 1 }
}
```

Example request (poll)

```
GET /api/v1/donors/me/donations?checkoutSessionId=cs_test_a1b2c3...
Authorization: Bearer <accessToken>
```

Response (poll, once recorded)

```
200 OK
{
  "success": true,
  "message": "Donation retrieved successfully",
  "data": {
    "donationID": 12,
    "donationCode": "DON-00012",
    "donationDate": "2026-09-14T16:20:00Z",
    "donationAmt": 50.00,
```

```
      "donationDesc": "For the winter blanket drive",
      "shelter": { "shelterID": 3, "shelterName": "Athens Shelter" }
    }
  }
```

Response (poll, not yet recorded)

```
200 OK
{
  "success": true,
  "message": "Donation retrieved successfully",
  "data": null
}
```

`donationAmt` is in US dollars, `donationDesc` is null when none was given, and `donationCode` is the display code staff see too. The Stripe IDs are never included, and the checkoutSessionId is only ever a lookup key.

Error Cases

Scenario	Error Code	HTTP Status
Invalid shelterID, dateFrom or dateTo; invalid page or limit	BAD_REQUEST	400
No/invalid token, or the account is Deactivated	UNAUTHORIZED	401
Role not permitted	FORBIDDEN	403

6.2 GET /api/v1/donors/me/donations/stats

Returns totals over all of the donor's donations, with a per-shelter breakdown. The list's filters do not apply.

Request

Field	Value
Method	GET
Auth required	Yes — Donor only
Query params	yearStart (ISO date-time, required — the client's local start of the current year)

The server does not know the donor's time zone, so the client sends its own local start of year and `thisYearAmount` counts donations dated on or after it. `byShelter` is ordered by `totalAmount`, largest first. A donor with no donations gets zeros and an empty `byShelter` list, not an error.

Example request

```
GET /api/v1/donors/me/donations/stats?yearStart=2026-01-01T00:00:00Z
Authorization: Bearer <accessToken>
```

Response

```
200 OK
{
  "success": true,
  "message": "Donation stats retrieved successfully",
  "data": {
    "totalAmount": 175.00,
    "donationCount": 4,
    "thisYearAmount": 125.00,
    "byShelter": [
```

```
      { "shelterID": 3, "shelterName": "Athens Shelter", "totalAmount":
125.00, "donationCount": 3 },
      { "shelterID": 5, "shelterName": "Decatur Shelter", "totalAmount":
50.00, "donationCount": 1 }
    ]
  }
}
```

Error Cases

Scenario	Error Code	HTTP Status
yearStart missing or not a valid date-time	BAD_REQUEST	400
No/invalid token, or the account is Deactivated	UNAUTHORIZED	401
Role not permitted	FORBIDDEN	403

6.3 GET /api/v1/donors/me/donations/:id

Returns one of the logged-in donor's own donations.

Request

Field	Value
Method	GET
Auth required	Yes — Donor only
Path params	id (int, required) — donationID

Example request

```
GET /api/v1/donors/me/donations/12
Authorization: Bearer <accessToken>
```

Response

```
200 OK
{
  "success": true,
  "message": "Donation retrieved successfully",
  "data": {
    "donationID": 12,
    "donationCode": "DON-00012",
    "donationDate": "2026-09-14T16:20:00Z",
    "donationAmt": 50.00,
    "donationDesc": "For the winter blanket drive",
    "shelter": { "shelterID": 3, "shelterName": "Athens Shelter" }
  }
}
```

The shape is the same as a history item (6.1). A donation that belongs to another donor answers 404, exactly as one that does not exist (2.4).

Error Cases

Scenario	Error Code	HTTP Status
id is not a positive integer	BAD_REQUEST	400
No/invalid token, or the account is Deactivated	UNAUTHORIZED	401

Scenario	Error Code	HTTP Status
Role not permitted	FORBIDDEN	403
Donation does not exist, or belongs to another donor	NOT_FOUND	404

7. Closing the Account

7.1 DELETE /api/v1/donors/me

Closes the donor's own account, either by deactivating it (data retained) or permanently deleting it. See Section 9 for exactly what each mode keeps.

Request

Field	Value
Method	DELETE
Auth required	Yes — Donor only
Body fields	mode (string, required — "deactivate" or "delete")

`deactivate` keeps the Donor row and sets `accountStatus` to Deactivated; the refresh token is cleared. `delete` removes the Donor and Users rows. Unlike other roles, nothing blocks a donor from leaving: there is no 409 in either mode, because a donor has no open tasks, appointments or applications to hand over. Every donation stays on the shelter's books.

Example request

```
DELETE /api/v1/donors/me
Authorization: Bearer <accessToken>
Content-Type: application/json
{ "mode": "deactivate" }
```

Response

```
200 OK
{
  "success": true,
  "message": "Account deactivated",
  "data": null
}
```

The message is "Account deleted" when mode is delete.

Error Cases

Scenario	Error Code	HTTP Status
No/invalid token, or the account is Deactivated	UNAUTHORIZED	401
Role not permitted	FORBIDDEN	403
mode missing or not "deactivate"/"delete"	VALIDATION_ERROR	422

8. Status & Derived Values

Several values the API returns are derived or fixed by rule. This section defines them so clients do not re-derive them.

8.1 How a donation is recorded

- **Amount.** The client sends whole US dollars (1–10000). The stored `donationAmt` is `amount_total / 100` from the paid Stripe session, so it is whatever Stripe actually charged.
- **Once only.** `stripeCheckoutSessionID` is unique, so a replayed webhook event cannot create a second donation.
- **Eventually consistent.** There is a short gap between a paid Checkout and the Donation row. That is why the confirmation page polls (6.1) rather than reading the result of POST /donations/checkout.
- **Abandoned checkouts.** A donor who cancels or never pays leaves no Donation row, so history and stats never include them.

8.2 Which shelters accept donations

Shelter status	POST /donations/checkout	GET /shelters
Open	Accepted	Returned by default and with acceptingDonations=true
Full	Accepted	Returned only with acceptingDonations=true
Closed	409 CONFLICT	Never returned

8.3 Stats windows

`totalAmount` and `donationCount` cover all of the donor's donations. `thisYearAmount` counts donations dated on or after the `yearStart` the client sent. `byShelter` groups by shelter, ordered by `totalAmount` descending. In the history list, `dateFrom` is inclusive and `dateTo` is exclusive, so adjacent ranges never double-count a donation.

9. Data Retention on Account Deletion

A donor can leave in two ways (7.1). Deactivating keeps everything; deleting removes the donor's identity but deliberately preserves their donations, because the money has already reached the shelter and the shelter's records must stay whole.

Data	On deactivate	On delete
Donor row	Kept — accountStatus set to Deactivated	Removed
Users row (login identity)	Kept	Removed
Refresh token	Cleared (forces logout)	Gone with the Users row
Donation rows	Unchanged	Kept
Donation.donorID	Unchanged	Set to NULL

- **"Former donor".** Wherever staff see a donor on a donation, the donor is null once the donor has deleted their account. Staff clients render "Former donor" (Section 10).
- **Nothing is refunded or removed.** Shelter totals, including staff stats, are the same before and after a donor deletes their account.
- **No donor can read them afterwards.** With the donor gone, no donor route can reach a null-donor donation; only staff and Admin can.

10. Related Public & Staff Endpoints

These endpoints are specified in 07-Public API Reference and 11-Staff API Reference. The table records only how they interact with the donor behaviour above.

Endpoint	Reference	What it does for donors
GET /shelters?acceptingDonations=true	07 — 1.5	The Donate page's shelter picker: Open and Full shelters. Without the flag only Open shelters are returned (8.2).
GET /donations	11 — 9.1	Staff list of donations received at the shelter, newest first. `donor` is null once the donor has deleted their account.
GET /donations/stats	11 — 9.2	Staff totals for the shelter, including donations from deleted donors.
GET /donations/:id	11 — 9.3	Staff detail of one donation. `donor` is null once the donor has deleted their account. The Stripe customer ID is never included.

> ***Note for 07 and 11:*** *the public shelter list (07, 1.5) does not yet document the acceptingDonations flag, and the Staff donation endpoints (11, 9.1 and 9.3) do not yet describe the null donor after deletion. Those rules are specified here (8.2 and Section 9) and the references should be updated to match.*

11. End-to-End Flow

One donor, from sign-up to a donation that outlives their account. Each step names the endpoint that performs it.

Flow — Donor sign-up to account deletion

1. Sign up: POST /auth/register with role "donor" and no shelter. The account is Active at once.
2. Optional onboarding: the wizard saves each step with PUT /donors/me then PATCH /donors/me/onboarding-step, and finishes with PATCH /donors/me/onboarding-complete. If the donor chooses "Skip for now", nothing is called; they can use the dashboard and donate for that browser session, and the next login returns to the wizard.
3. Choose shelter and amount: the Donate page lists shelters from GET /shelters?acceptingDonations=true, and the donor enters a whole-dollar amount and an optional message.
4. Check out: POST /donations/checkout returns a checkoutUrl (201); the donor pays on Stripe Checkout. No Donation row exists yet.
5. Webhook: Stripe calls POST /webhooks/stripe with checkout.session.completed and metadata.kind "donation"; the paid session is recorded as one Donation for the amount Stripe charged. A replayed event is ignored.
6. Confirmation poll: Stripe returns the donor to /donor/donate/confirmation?session_id=..., which polls GET /donors/me/donations?checkoutSessionId=.... It receives data: null until the webhook lands, then the donation.
7. History and stats: GET /donors/me/donations lists the donation, GET /donors/me/donations/stats?yearStart=... updates the totals, and GET /donors/me/donations/:id shows it in detail.
8. Staff at the shelter see the donation in GET /donations with the donor's name.
9. Account deletion: DELETE /donors/me with { mode: "delete" } succeeds with no blockers. The Donor and Users rows are removed, but the donation stays on the shelter's books with donorID null, and staff now see it as from a "Former donor".